

Start coding or [generate](#) with AI.

Machine Learning Experiment Number 3

Aim : To implement a Multiple Linear Regression model from scratch using NumPy matrix operations to predict the selling price of used cars using multiple features from the CarDekho Used Car Dataset.

Learning Outcomes • Understand the working principle of Multiple Linear Regression.

- Perform data preprocessing and feature engineering.
- Convert categorical data into numerical features.
- Standardize numerical features.
- Implement the Ordinary Least Squares method using matrix operations.
- Train and test a regression model without using a ready-made regression function.
- Evaluate and interpret regression performance using RSS and R^2 .
- Understand the effect of individual features on the predicted price.

Dataset

Use the **CarDekho Used Car Dataset** (`car_data.csv`). [CarDekho Used Car Dataset](#)

Explain meaning of each attribute in this dataset in below comment section :

Here, **Selling_Price** is the target variable.

Software/Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- NumPy
- Matplotlib

Task 1: Set Up the Python Environment**

- Create a new Google Colab notebook.
- Import **NumPy** for numerical and matrix operations.
- Import **Matplotlib** for data visualization.

- Do not use ready-made machine-learning regression functions such as `LinearRegression()` from Scikit-learn.
- The regression model must be implemented using matrix operations.

```
# Task 1: Set Up the Python Environment

# Import NumPy for numerical and matrix operations
import numpy as np

# Import Matplotlib for data visualization
import matplotlib.pyplot as plt

# Check installed versions
print("NumPy version:", np.__version__)
print("Matplotlib version:", plt.matplotlib.__version__)

print("Python environment setup completed successfully!")
```

```
NumPy version: 2.1.3
Matplotlib version: 3.10.0
Python environment setup completed successfully!
```

✓ Task 2: Load and Inspect the Dataset

- Download the `car_data.csv` file.
- Upload the dataset to Google Colab.
- Load the dataset into Python.
- Display the first few records.
- Determine the number of rows and columns.
- Identify the numerical and categorical attributes.
- Separate `Selling_Price` as the target variable.

```
# TASK 2: LOAD AND INSPECT THE DATASET

import pandas as pd
from google.colab import files

# Upload the CSV file
uploaded = files.upload()

# Get the uploaded file name
file_name = list(uploaded.keys())[0]

# Load the dataset
df = pd.read_csv(file_name)

# Remove extra spaces from column names
df.columns = df.columns.str.strip()
```

```
# Display the first 5 records
print("First 5 records:")
display(df.head())

# Display column names
print("\nColumn names in the dataset:")
print(df.columns.tolist())

# Number of rows and columns
print("\nNumber of rows:", df.shape[0])
print("Number of columns:", df.shape[1])

# Identify numerical attributes
numerical_attributes = df.select_dtypes(include=["number"]).columns.tolist()

# Identify categorical attributes
categorical_attributes = df.select_dtypes(include=["object", "category"]).columns.tolist()

print("\nNumerical attributes:")
print(numerical_attributes)

print("\nCategorical attributes:")
print(categorical_attributes)

# Find Selling_Price column
target_column = None

for column in df.columns:
    if column.lower().replace(" ", "_") == "selling_price":
        target_column = column
        break

# Check whether Selling_Price exists
if target_column is None:
    print("\nERROR: Selling_Price column was not found.")
    print("Please check the column names displayed above.")
else:
    # Separate target variable
    target = df[target_column]

    # Separate input features
    features = df.drop(columns=[target_column])

    print("\nTarget variable:", target_column)
    print("Target shape:", target.shape)

    print("\nFeature data:")
    display(features.head())

    print("\nFeature shape:", features.shape)

    print("\nTask 2 completed successfully!")
```

Choose Files

No file chosen

Upload widget is only available when the cell has

been executed in the current browser session. Please rerun this cell to enable.

Saving cardekho_dataset.csv to cardekho_dataset (4).csv

First 5 records:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual
3	3	Maruti Alto	Maruti	Alto	9	37000	Individual
4	4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer

Column names in the dataset:

['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'sel

Number of rows: 15411

Number of columns: 14

Numerical attributes:

['Unnamed: 0', 'vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power',

Categorical attributes:

['car_name', 'brand', 'model', 'seller_type', 'fuel_type', 'transmission_type

Target variable: selling_price

Target shape: (15411,)

Feature data:

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual

✓ Task 3: Check and Handle Missing Data

- Check every attribute for missing or null values.
- Count the number of missing values in each column.
- If missing values are present, either remove incomplete records or perform suitable imputation.

- Verify that the final dataset does not contain missing values.
- Record the preprocessing method used.

```
# TASK 3: CHECK AND HANDLE MISSING DATA

# Step 1: Check missing values in each column
print("Missing values before preprocessing:")
print(df.isnull().sum())

# Step 2: Calculate total missing values
total_missing = df.isnull().sum().sum()

print("\nTotal missing values:", total_missing)

# Step 3: Handle missing values
# Numerical columns -> replace missing values with median
numerical_columns = df.select_dtypes(include=["number"]).columns

for column in numerical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].median())

# Categorical columns -> replace missing values with mode
categorical_columns = df.select_dtypes(include=["object", "category"]).columns

for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].mode()[0])

# Step 4: Verify that no missing values remain
print("\nMissing values after preprocessing:")
print(df.isnull().sum())

# Step 5: Final verification
remaining_missing = df.isnull().sum().sum()

print("\nTotal missing values after preprocessing:", remaining_missing)

if remaining_missing == 0:
    print("SUCCESS: The dataset contains no missing values.")
else:
    print("WARNING: Missing values are still present.")

# Step 6: Record the preprocessing method
print("\nPreprocessing method used:")
print("- Numerical missing values: replaced with the median.")
print("- Categorical missing values: replaced with the mode.")
print("- No incomplete records were removed.")
```

```
Missing values before preprocessing:
Unnamed: 0      0
car_name      0
brand         0
```

```

model          0
vehicle_age    0
km_driven      0
seller_type    0
fuel_type      0
transmission_type 0
mileage        0
engine         0
max_power      0
seats          0
selling_price  0
dtype: int64

```

Total missing values: 0

Missing values after preprocessing:

```

Unnamed: 0      0
car_name        0
brand           0
model           0
vehicle_age     0
km_driven       0
seller_type     0
fuel_type       0
transmission_type 0
mileage         0
engine          0
max_power       0
seats           0
selling_price   0
dtype: int64

```

Total missing values after preprocessing: 0

SUCCESS: The dataset contains no missing values.

Preprocessing method used:

- Numerical missing values: replaced with the median.
- Categorical missing values: replaced with the mode.
- No incomplete records were removed.

✓ Task 4: Select and Prepare Features

- Examine all available attributes.
- Decide how `Car_Name` should be handled.
- Select meaningful predictor variables for the regression model.
- Create the predictor matrix X and target vector y .
- Record the final list of features selected for the model.

```
# TASK 4: SELECT AND PREPARE FEATURES
```

```

# Display all available attributes
print("Available attributes:")
print(df.columns.tolist())

```

```
# Make column names consistent
df.columns = df.columns.str.strip()

# Find the target column safely
target_column = None

for column in df.columns:
    if column.lower().replace(" ", "_") == "selling_price":
        target_column = column
        break

if target_column is None:
    raise ValueError("Selling_Price column was not found in the dataset.")

# Handle Car_Name
# Car_Name is removed because it is a high-cardinality categorical
# attribute and is not required for this regression model.
car_name_column = None

for column in df.columns:
    if column.lower().replace(" ", "_") == "car_name":
        car_name_column = column
        break

if car_name_column is not None:
    df = df.drop(columns=[car_name_column])

# Select meaningful predictor variables
feature_columns = [
    column for column in df.columns
    if column != target_column
]

# Check that all selected features are numerical
non_numeric_features = df[feature_columns].select_dtypes(
    exclude=["number"]
).columns.tolist()

if len(non_numeric_features) > 0:
    print("\nNon-numerical features found:")
    print(non_numeric_features)

    # Convert categorical features into numerical dummy variables
    df = pd.get_dummies(
        df,
        columns=non_numeric_features,
        drop_first=True
    )

# Update feature list after encoding
feature_columns = [
    column for column in df.columns
    if column != target_column
]
```

```

# Create predictor matrix X
X = df[feature_columns].astype(float).values

# Create target vector y
y = df[target_column].astype(float).values

# Display results
print("\nCar_Name handling:")
if car_name_column is not None:
    print("Car_Name was removed from the model.")
else:
    print("Car_Name was not present in the dataset.")

print("\nFinal features selected:")
for feature in feature_columns:
    print("-", feature)

print("\nPredictor matrix X shape:", X.shape)
print("Target vector y shape:", y.shape)

print("\nFirst 5 rows of X:")
print(X[:5])

print("\nFirst 5 values of y:")
print(y[:5])

print("\nTask 4 completed successfully!")

```

Available attributes:
 ['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven', 'seats']

Non-numerical features found:
 ['brand', 'model', 'seller_type', 'fuel_type', 'transmission_type']

Car_Name handling:
 Car_Name was removed from the model.

Final features selected:

- Unnamed: 0
- vehicle_age
- km_driven
- mileage
- engine
- max_power
- seats
- brand_BMW
- brand_Bentley
- brand_Datsun
- brand_Ferrari
- brand_Force
- brand_Ford
- brand_Honda
- brand_Hyundai
- brand_ISUZU
- brand_Isuzu
- brand_Jaguar
- brand_Jeep

```
- brand_Kia  
- brand_Land Rover  
- brand_Lexus  
- brand_MG  
- brand_Mahindra  
- brand_Maruti  
- brand_Maserati  
- brand_Mercedes-AMG  
- brand_Mercedes-Benz  
- brand_Mini  
- brand_Nissan  
- brand_Porsche  
- brand_Renault  
- brand_Rolls-Royce  
- brand_Skoda  
- brand_Tata  
- brand_Toyota  
- brand_Volkswagen  
- brand_Volvo  
- model_5  
- model_6  
- model_7  
- model_A4  
- model_A6  
- model_A8  
- model_Alto  
- model_Altroz  
- model_Alturas
```

✓ Task 5: Encode Categorical Variables

Identify the categorical variables:

- Fuel_Type
- Seller_Type
- Transmission

Perform the following:

- Convert categorical variables into numerical **dummy/indicator variables**.
- Verify that all predictors are numerical after encoding.
- Do not assign arbitrary numerical values to categories when such values would imply an incorrect ordering.
- Display the transformed dataset.

```
# TASK 5: ENCODE CATEGORICAL VARIABLES
```

```
# Make a copy of the dataset  
df_encoded = df.copy()
```

```
# Identify categorical variables  
categorical_columns = [  
    "Fuel_Type",
```

```
"Seller_Type",
"Transmission"
]

# Check which categorical columns are actually present
available_categorical_columns = [
    column for column in categorical_columns
    if column in df_encoded.columns
]

print("Categorical variables found:")
print(available_categorical_columns)

# Convert categorical variables into dummy/indicator variables
df_encoded = pd.get_dummies(
    df_encoded,
    columns=available_categorical_columns,
    drop_first=True,
    dtype=int
)

# Display the transformed dataset
print("\nTransformed dataset:")
display(df_encoded.head())

# Verify that all columns are numerical
non_numeric_columns = df_encoded.select_dtypes(
    exclude=["number"]
).columns.tolist()

print("\nNon-numerical columns after encoding:")
print(non_numeric_columns)

if len(non_numeric_columns) == 0:
    print("\nSUCCESS: All predictors are numerical after encoding.")
else:
    print("\nWARNING: Some non-numerical columns still exist.")

# Display final column names
print("\nFinal columns after encoding:")
print(df_encoded.columns.tolist())

# Create final predictor matrix X and target vector y
target_column = "Selling_Price"

# Handle possible column-name variation
if target_column not in df_encoded.columns:
    matching_columns = [
        column for column in df_encoded.columns
        if column.lower().replace(" ", "_") == "selling_price"
    ]

    if len(matching_columns) > 0:
        target_column = matching_columns[0]
    else:
```

```

        raise ValueError("Selling_Price column was not found.")

X = df_encoded.drop(columns=[target_column]).astype(float).values
y = df_encoded[target_column].astype(float).values

print("\nFinal predictor matrix X shape:", X.shape)
print("Final target vector y shape:", y.shape)

print("\nTask 5 completed successfully!")

```

Categorical variables found:
[]

Transformed dataset:

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats	selling_price
0	0	9	120000	19.70	796	46.30	5	1500000
1	1	5	20000	18.90	1197	82.00	5	1200000
2	2	11	60000	17.00	1197	80.00	5	1000000
3	3	9	37000	20.92	998	67.10	5	800000
4	4	6	30000	22.77	1498	98.59	5	700000

5 rows × 165 columns

Non-numerical columns after encoding:
['brand_BMW', 'brand_Bentley', 'brand_Datsun', 'brand_Ferrari', 'brand_Force']

WARNING: Some non-numerical columns still exist.

Final columns after encoding:
['Unnamed: 0', 'vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power',

Final predictor matrix X shape: (15411, 164)
Final target vector y shape: (15411,)

Task 5 completed successfully!

✓ Task 6: Perform Feature Engineering

Create a new feature called `Car_Age`.

Calculate it using:

[`Car_Age = Current Year - Year`]

Perform the following:

- Create the `Car_Age` column.
- Display several values of the newly created feature.
- Verify that the calculated ages are reasonable.

- Decide whether the original `Year` feature should be retained.
- Provide a reason for your decision.

```
# TASK 6: FEATURE ENGINEERING - CREATE CAR_AGE

import pandas as pd
from datetime import datetime

# Load the dataset again
# If df already exists, use it
if "df" not in globals():
    from google.colab import files

    uploaded = files.upload()
    file_name = list(uploaded.keys())[0]
    df = pd.read_csv(file_name)

# Clean column names
df.columns = df.columns.str.strip()

# Find the Year column
year_column = None

for column in df.columns:
    if column.strip().lower() == "year":
        year_column = column
        break

# Check whether Year exists
if year_column is None:
    print("ERROR: The 'Year' column was not found.")
    print("Available columns are:")
    print(df.columns.tolist())
else:
    # Convert Year to numeric
    df[year_column] = pd.to_numeric(df[year_column], errors="coerce")

    # Remove rows where Year is missing
    df = df.dropna(subset=[year_column]).copy()

    # Convert Year to integer
    df[year_column] = df[year_column].astype(int)

    # Get current year
    current_year = datetime.now().year

    # Create Car_Age
    df["Car_Age"] = current_year - df[year_column]

    # Display the new feature
    print("Current Year:", current_year)
    print("\nYear and Car_Age:")
    display(df[[year_column, "Car_Age"]].head(10))
```

```

# Check Car_Age values
print("\nCar_Age statistics:")
print(df["Car_Age"].describe())

# Verify that ages are reasonable
if (df["Car_Age"] < 0).any():
    print("\nWARNING: Some Car_Age values are negative.")
elif (df["Car_Age"] > 100).any():
    print("\nWARNING: Some Car_Age values are greater than 100.")
else:
    print("\nSUCCESS: Car_Age values are reasonable.")

# Remove original Year column
df = df.drop(columns=[year_column])

print("\nOriginal Year column removed.")
print("Reason: Car_Age provides the same time information in a more")
print("meaningful and interpretable form.")

# Display final dataset
print("\nDataset after feature engineering:")
display(df.head())

print("\nTask 6 completed successfully!")

```

```

ERROR: The 'Year' column was not found.
Available columns are:
['Unnamed: 0', 'vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power',

```

✓ Task 7: Perform Exploratory Data Analysis

Perform basic exploratory data analysis using Matplotlib.

- Create a scatter plot of `Present_Price` versus `Selling_Price`.
- Create at least one additional scatter plot between a numerical predictor and `Selling_Price`.
- Label the axes appropriately.
- Add suitable titles to the plots.
- Observe the direction and approximate strength of the relationships.
- Record your observations.

```

# TASK 7: EXPLORATORY DATA ANALYSIS

import pandas as pd
import matplotlib.pyplot as plt

# Make sure the required columns exist
required_columns = ["Present_Price", "Selling_Price"]

```

```
missing_columns = [
    column for column in required_columns
    if column not in df.columns
]

if len(missing_columns) > 0:
    print("ERROR: The following required columns are missing:")
    print(missing_columns)
    print("\nAvailable columns:")
    print(df.columns.tolist())
else:
    # Convert required columns to numeric
    df["Present_Price"] = pd.to_numeric(
        df["Present_Price"], errors="coerce"
    )

    df["Selling_Price"] = pd.to_numeric(
        df["Selling_Price"], errors="coerce"
    )

    # Remove rows with missing values in required columns
    plot_data = df.dropna(
        subset=["Present_Price", "Selling_Price"]
    ).copy()

    # -----
    # PLOT 1: Present_Price vs Selling_Price
    # -----

    plt.figure(figsize=(8, 5))

    plt.scatter(
        plot_data["Present_Price"],
        plot_data["Selling_Price"],
        color="blue",
        alpha=0.7
    )

    plt.xlabel("Present Price")
    plt.ylabel("Selling Price")
    plt.title("Present Price vs Selling Price")
    plt.grid(True, alpha=0.3)
    plt.show()

    # -----
    # Find another numerical predictor
    # -----

    numerical_columns = plot_data.select_dtypes(
        include=["number"]
    ).columns.tolist()

    # Remove target and Present_Price
    possible_predictors = [
```

```
        column for column in numerical_columns
        if column not in ["Selling_Price", "Present_Price"]
    ]

if len(possible_predictors) == 0:
    print("No additional numerical predictor is available.")
else:
    # Select the first available numerical predictor
    second_predictor = possible_predictors[0]

    # -----
    # PLOT 2: Second numerical predictor vs Selling_Price
    # -----

    plt.figure(figsize=(8, 5))

    plt.scatter(
        plot_data[second_predictor],
        plot_data["Selling_Price"],
        color="green",
        alpha=0.7
    )

    plt.xlabel(second_predictor)
    plt.ylabel("Selling Price")
    plt.title(
        second_predictor + " vs Selling Price"
    )
    plt.grid(True, alpha=0.3)
    plt.show()

    # -----
    # Calculate correlations
    # -----

    correlation_1 = plot_data[
        "Present_Price"
    ].corr(plot_data["Selling_Price"])

    correlation_2 = plot_data[
        second_predictor
    ].corr(plot_data["Selling_Price"])

    print("\nCorrelation results:")
    print(
        "Present_Price vs Selling_Price:",
        round(correlation_1, 3)
    )
    print(
        second_predictor + " vs Selling_Price:",
        round(correlation_2, 3)
    )

    print("\nTask 7 completed successfully!")
```

```
ERROR: The following required columns are missing:
['Present_Price', 'Selling_Price']
```

```
Available columns:
['Unnamed: 0', 'vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power',
```

✓ Task 8: Standardize Numerical Features

Identify the numerical predictors that need to be standardized.

Use the following standardization equation:

$$[z = (x - \mu) / \sigma]$$

where:

- x = original feature value
- μ = mean of the feature
- σ = standard deviation of the feature

Perform the following:

- Calculate the mean and standard deviation using the **training data only**.
- Standardize the training features.
- Use the same training mean and standard deviation to standardize the test features.
- Do not standardize the target variable `Selling_Price`.

```
# TASK 8: STANDARDIZE NUMERICAL FEATURES

import numpy as np
import pandas as pd

# -----
# STEP 1: Prepare X and y
# -----

# Make a copy of the current dataset
data = df.copy()

# Clean column names
data.columns = data.columns.str.strip()

# Find Selling_Price column
target_column = None

for column in data.columns:
    if column.lower().replace(" ", "_") == "selling_price":
        target_column = column
        break
```

```
if target_column is None:
    raise ValueError("Selling_Price column was not found.")

# Make sure Selling_Price is numeric
data[target_column] = pd.to_numeric(
    data[target_column],
    errors="coerce"
)

# Remove rows with missing target values
data = data.dropna(subset=[target_column]).copy()

# Separate predictors and target
X = data.drop(columns=[target_column])
y = data[target_column].astype(float)

# Convert categorical variables to dummy variables
categorical_columns = X.select_dtypes(
    include=["object", "category"]
).columns.tolist()

if len(categorical_columns) > 0:
    X = pd.get_dummies(
        X,
        columns=categorical_columns,
        drop_first=True,
        dtype=int
    )

# Convert all predictors to numeric
X = X.apply(pd.to_numeric, errors="coerce")

# Handle any missing predictor values
X = X.fillna(X.median())

# -----
# STEP 2: Split into training and testing data
# -----

np.random.seed(42)

# Create shuffled indices
indices = np.random.permutation(len(X))

# Use 80% for training and 20% for testing
train_size = int(0.80 * len(X))

train_indices = indices[:train_size]
test_indices = indices[train_size:]

X_train = X.iloc[train_indices].copy()
X_test = X.iloc[test_indices].copy()

y_train = y.iloc[train_indices].copy()
```

```
y_test = y.iloc[test_indices].copy()

# -----
# STEP 3: Identify numerical predictors
# -----

numerical_features = X_train.select_dtypes(
    include=["number"]
).columns.tolist()

print("Numerical predictors to be standardized:")
print(numerical_features)

# -----
# STEP 4: Calculate mean and standard deviation
#         USING TRAINING DATA ONLY
# -----

train_mean = X_train[numerical_features].mean()
train_std = X_train[numerical_features].std(ddof=0)

# Avoid division by zero for constant columns
train_std = train_std.replace(0, 1)

# -----
# STEP 5: Standardize training features
# -----

X_train_scaled = X_train.copy()

X_train_scaled[numerical_features] = (
    X_train[numerical_features] - train_mean
) / train_std

# -----
# STEP 6: Standardize test features
#         USING THE TRAINING MEAN AND STD
# -----

X_test_scaled = X_test.copy()

X_test_scaled[numerical_features] = (
    X_test[numerical_features] - train_mean
) / train_std

# -----
# STEP 7: Display results
# -----

print("\nTraining data shape:", X_train_scaled.shape)
print("Testing data shape:", X_test_scaled.shape)

print("\nFirst 5 rows of standardized training features:")
display(X_train_scaled.head())
```

```
print("\nFirst 5 rows of standardized test features:")
display(X_test_scaled.head())

# -----
# STEP 8: Verify standardization
# -----

print("\nTraining means after standardization:")
print(X_train_scaled[numerical_features].mean().round(6))

print("\nTraining standard deviations after standardization:")
print(X_train_scaled[numerical_features].std(ddof=0).round(6))

# -----
# STEP 9: Verify target was NOT standardized
# -----

print("\nTarget variable was NOT standardized.")
print("Original training target values:")
print(y_train.head().values)

print("\nOriginal testing target values:")
print(y_test.head().values)

print("\nTask 8 completed successfully!")
```

Numerical predictors to be standardized:

```
['Unnamed: 0', 'vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power',
```

Training data shape: (12328, 164)

Testing data shape: (3083, 164)

First 5 rows of standardized training features:

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats
3334	-0.978386	1.976831	0.317732	0.160198	-0.555761	-0.505726	-0.402
10928	0.727471	-0.673275	0.042356	1.844986	-0.457357	-0.620363	-0.402
2518	-1.163716	0.320515	0.739976	0.258457	-0.457357	-0.275051	2.067
11322	0.812082	-1.667065	-0.939819	-0.309529	0.025011	0.440606	-0.402
9394	0.402125	1.645568	0.115790	0.002025	-1.329480	-1.268407	-0.402

5 rows × 164 columns

First 5 rows of standardized test features:

	Unnamed: 0	vehicle_age	km_driven	mileage	engine	max_power	seats
15384	1.721743	-1.004538	-0.563471	-2.322647	1.279169	1.625569	-0.402
147	-1.697759	0.651778	0.409524	-0.752894	0.931864	1.099413	-0.402
1668	-1.354180	0.320515	0.124969	1.320137	-0.457357	-0.620363	-0.402
11280	0.803763	-0.342012	-0.563471	1.348896	-0.553831	-0.550177	0.832
13650	1.333026	-1.004538	-0.618547	-0.357461	-0.551902	-0.276454	-0.402

5 rows × 164 columns

Training means after standardization:

```
Unnamed: 0    0.0
vehicle_age    0.0
km_driven      0.0
mileage       -0.0
engine         0.0
max_power     0.0
seats        -0.0
dtype: float64
```

Training standard deviations after standardization:

```
Unnamed: 0    1.0
vehicle_age    1.0
km_driven      1.0
mileage        1.0
engine         1.0
max_power     1.0
seats          1.0
dtype: float64
```

Target variable was NOT standardized.

Original training target values:

```
[ 190000.  600000.  665000. 1570000. 160000. ]
```

Task 9: Split the Dataset

Original testing target values:

[4450000. 1290000. 485000. 400000. 800000.]

Divide the dataset into training and testing subsets.

Task 8 completed successfully!

- Use an **80:20 train-test ratio**.
- Randomly select the observations.
- Set a fixed random seed to make the experiment reproducible.
- Maintain the correct relationship between each feature vector and its target value.
- Display the number of observations in the training and testing sets.

```
# TASK 9: SPLIT THE DATASET

import numpy as np

# Make sure X and y exist
if "X" not in globals() or "y" not in globals():
    raise ValueError("X and y are not available. Please run the previous tas

# Convert X and y to NumPy arrays
X = np.asarray(X)
y = np.asarray(y)

# Check that X and y have the same number of observations
if len(X) != len(y):
    raise ValueError("X and y do not contain the same number of observations

# Set a fixed random seed for reproducibility
np.random.seed(42)

# Create randomly shuffled indices
indices = np.random.permutation(len(X))

# Calculate 80% training size
train_size = int(0.80 * len(X))

# Split the indices
train_indices = indices[:train_size]
test_indices = indices[train_size:]

# Create training and testing datasets
X_train = X[train_indices]
X_test = X[test_indices]

y_train = y[train_indices]
y_test = y[test_indices]

# Display the results
print("TASK 9: TRAIN-TEST SPLIT")
print("-" * 40)
```

```

print("Total observations:", len(X))
print("Training observations:", len(X_train))
print("Testing observations:", len(X_test))

print("\nTraining percentage:",
      round(len(X_train) / len(X) * 100, 2), "%")

print("Testing percentage:",
      round(len(X_test) / len(X) * 100, 2), "%")

print("\nTraining feature shape:", X_train.shape)
print("Testing feature shape:", X_test.shape)

print("\nTraining target shape:", y_train.shape)
print("Testing target shape:", y_test.shape)

print("\nRandom seed used: 42")

print("\nTask 9 completed successfully!")

```

TASK 9: TRAIN-TEST SPLIT

```

-----
Total observations: 15411
Training observations: 12328
Testing observations: 3083

Training percentage: 79.99 %
Testing percentage: 20.01 %

Training feature shape: (12328, 164)
Testing feature shape: (3083, 164)

Training target shape: (12328,)
Testing target shape: (3083,)

Random seed used: 42

Task 9 completed successfully!

```

✓ Task 10: Construct the Feature Matrix

Construct the matrices required for regression.

Create:

- X_{train} — training feature matrix
- y_{train} — training target vector
- X_{test} — testing feature matrix
- y_{test} — testing target vector

Verify:

- Number of rows in each matrix.
- Number of predictor columns.

- Shape of the training and testing matrices.

```
# TASK 10: CONSTRUCT THE FEATURE MATRICES

import numpy as np

# Check that Task 9 has been completed
if "X_train" not in globals() or "X_test" not in globals():
    raise ValueError("Please run Task 9 first to create X_train and X_test.")

if "y_train" not in globals() or "y_test" not in globals():
    raise ValueError("Please run Task 9 first to create y_train and y_test.")

# Convert all data to NumPy arrays
Xtrain = np.asarray(X_train, dtype=float)
ytrain = np.asarray(y_train, dtype=float)

Xtest = np.asarray(X_test, dtype=float)
ytest = np.asarray(y_test, dtype=float)

# Make sure target vectors are one-dimensional
ytrain = ytrain.reshape(-1)
ytest = ytest.reshape(-1)

# -----
# VERIFY THE MATRICES
# -----

print("TASK 10: FEATURE MATRIX CONSTRUCTION")
print("-" * 45)

print("\nXtrain (Training Feature Matrix)")
print("Shape:", Xtrain.shape)
print("Rows:", Xtrain.shape[0])
print("Predictor columns:", Xtrain.shape[1])

print("\nytrain (Training Target Vector)")
print("Shape:", ytrain.shape)
print("Rows:", ytrain.shape[0])

print("\nXtest (Testing Feature Matrix)")
print("Shape:", Xtest.shape)
print("Rows:", Xtest.shape[0])
print("Predictor columns:", Xtest.shape[1])

print("\nytest (Testing Target Vector)")
print("Shape:", ytest.shape)
print("Rows:", ytest.shape[0])

# -----
# CHECK THAT THE MATRICES MATCH CORRECTLY
# -----
```

```

if Xtrain.shape[0] != ytrain.shape[0]:
    raise ValueError("Xtrain and ytrain have different numbers of rows.")

if Xtest.shape[0] != ytest.shape[0]:
    raise ValueError("Xtest and ytest have different numbers of rows.")

if Xtrain.shape[1] != Xtest.shape[1]:
    raise ValueError("Xtrain and Xtest have different numbers of predictor c

print("\nVerification:")
print("✓ Xtrain rows match ytrain rows")
print("✓ Xtest rows match ytest rows")
print("✓ Xtrain and Xtest have the same number of predictors")

print("\nFirst 5 rows of Xtrain:")
print(Xtrain[:5])

print("\nFirst 5 values of ytrain:")
print(ytrain[:5])

print("\nFirst 5 rows of Xtest:")
print(Xtest[:5])

print("\nFirst 5 values of ytest:")
print(ytest[:5])

print("\nTask 10 completed successfully!")

```

TASK 10: FEATURE MATRIX CONSTRUCTION

Xtrain (Training Feature Matrix)
 Shape: (12328, 164)
 Rows: 12328
 Predictor columns: 164

ytrain (Training Target Vector)
 Shape: (12328,)
 Rows: 12328

Xtest (Testing Feature Matrix)
 Shape: (3083, 164)
 Rows: 3083
 Predictor columns: 164

ytest (Testing Target Vector)
 Shape: (3083,)
 Rows: 3083

Verification:
 ✓ Xtrain rows match ytrain rows
 ✓ Xtest rows match ytest rows
 ✓ Xtrain and Xtest have the same number of predictors

First 5 rows of Xtrain:
 [[4.2590e+03 1.2000e+01 7.3000e+04 2.0360e+01 1.1970e+03 7.8900e+01
 5.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00


```
import numpy as np

# Check that Task 10 has been completed
if "Xtrain" not in globals() or "Xtest" not in globals():
    raise ValueError("Please run Task 10 first.")

# Make sure the feature matrices are NumPy arrays
Xtrain = np.asarray(Xtrain, dtype=float)
Xtest = np.asarray(Xtest, dtype=float)

# Create a column of 1.0 for the bias/intercept term
bias_train = np.ones((Xtrain.shape[0], 1))
bias_test = np.ones((Xtest.shape[0], 1))

# Add the bias column to the beginning of the matrices
Xtrain_bias = np.hstack((bias_train, Xtrain))
Xtest_bias = np.hstack((bias_test, Xtest))

# -----
# VERIFY THE NEW MATRIX DIMENSIONS
# -----

print("TASK 11: ADD BIAS / INTERCEPT TERM")
print("-" * 45)

print("\nOriginal Xtrain shape:", Xtrain.shape)
print("New Xtrain_bias shape:", Xtrain_bias.shape)

print("\nOriginal Xtest shape:", Xtest.shape)
print("New Xtest_bias shape:", Xtest_bias.shape)

# Verify that the first column contains only 1.0
print("\nFirst 5 values of the bias column in training data:")
print(Xtrain_bias[:5, 0])

print("\nFirst 5 values of the bias column in testing data:")
print(Xtest_bias[:5, 0])

# Verify the bias column
if np.all(Xtrain_bias[:, 0] == 1.0) and np.all(Xtest_bias[:, 0] == 1.0):
    print("\nSUCCESS: Bias column added correctly.")
else:
    raise ValueError("Bias column was not created correctly.")

# Display the first 5 rows
print("\nFirst 5 rows of Xtrain_bias:")
print(Xtrain_bias[:5])

print("\nFirst 5 rows of Xtest_bias:")
print(Xtest_bias[:5])

print("\nThe first column represents the intercept ( $\beta_0$ ).")
print("All remaining columns represent the predictor variables.")

print("\nTask 11 completed successfully!")
```

[illegible]

Task 12: Implement the OLS Function

Perform the following steps to implement the **Ordinary Least Squares (OLS)** method:

- Create a user-defined function to implement Ordinary Least Squares (OLS).
- Calculate the matrix $X^T X$.
- Calculate the matrix $X^T y$.
- Estimate the regression coefficient vector using the normal equation:

$$\beta = (X^T X)^{-1} X^T y.$$

- Return the calculated coefficient vector.
- Use **NumPy matrix operations** to perform the calculations.
- Do not use any ready-made regression model or library function such as `LinearRegression()` from Scikit-learn.

```
# TASK 12: IMPLEMENT ORDINARY LEAST SQUARES (OLS)
# =====

import numpy as np

# -----
# STEP 1: Check that training data exists
# -----

if "X_train" not in globals():
    raise ValueError("X_train was not found. Please run Task 9 first.")

if "y_train" not in globals():
    raise ValueError("y_train was not found. Please run Task 9 first.")

# Convert training data to NumPy arrays
X_train_data = np.asarray(X_train, dtype=float)
y_train_data = np.asarray(y_train, dtype=float).reshape(-1)

# -----
# STEP 2: Check dimensions
# -----

if X_train_data.shape[0] != y_train_data.shape[0]:
    raise ValueError(
        "The number of rows in X_train and y_train must be the same."
    )

# -----
# STEP 3: Add the intercept/bias column
# -----

ones_column = np.ones((X_train_data.shape[0], 1))
```

```
Xtrain_bias = np.hstack((ones_column, X_train_data))

# -----
# STEP 4: Define the OLS function
# -----

def OLS(X, y):
    """
    Ordinary Least Squares using matrix operations.

    Formula:
        
$$\beta = (X^T X)^{-1} X^T y$$

    """

    # Calculate X transpose
    XT = X.T

    # Calculate  $X^T X$ 
    XTX = XT @ X

    # Calculate  $X^T y$ 
    XTy = XT @ y

    # Calculate beta
    # pinv is used to handle cases where  $X^T X$  is singular.
    beta = np.linalg.pinv(XTX) @ XTy

    return beta

# -----
# STEP 5: Calculate regression coefficients
# -----

beta = OLS(Xtrain_bias, y_train_data)

# -----
# STEP 6: Display results
# -----

print("TASK 12: ORDINARY LEAST SQUARES (OLS)")
print("=" * 50)

print("\nTraining feature matrix shape:")
print(Xtrain_bias.shape)

print("\nTraining target vector shape:")
print(y_train_data.shape)

# Calculate  $X^T X$  and  $X^T y$  for display
XTX = Xtrain_bias.T @ Xtrain_bias
XTy = Xtrain_bias.T @ y_train_data

print("\n $X^T X$  shape:")
print(XTX.shape)
```

```

print("\nX^T y shape:")
print(XTy.shape)

print("\nRegression coefficient vector beta:")
print(beta)

print("\nNumber of coefficients:")
print(len(beta))

print("\nIntercept (beta_0):")
print(beta[0])

print("\nTask 12 completed successfully!")

```

TASK 12: ORDINARY LEAST SQUARES (OLS)

=====

Training feature matrix shape:
(12328, 165)

Training target vector shape:
(12328,)

$X^T X$ shape:
(165, 165)

$X^T y$ shape:
(165,)

Regression coefficient vector beta:

```

[ 3.01031689e+06  3.30629125e-01 -7.96388930e+04 -4.93254658e-01
 -2.41772831e+03 -1.59695735e+02  5.85277757e+03 -3.98549198e+04
 -1.20137590e+06  4.89493247e+06 -1.67337750e+06  1.69441612e+07
 -8.14281241e+05 -1.51673825e+06 -1.60362667e+06 -1.72691659e+06
 -4.45087379e-04 -8.30555215e+05  2.10034293e+05  5.07856354e+03
 -6.01461496e+05  2.67683946e+05  1.70508466e+06 -7.48594283e+05
 -1.54687899e+06 -1.91408004e+06  1.27812321e+06  4.40881150e+05
  2.80368839e+05 -3.62561864e+05 -1.27761740e+06  1.38139108e+06
 -1.58098691e+06 -2.27982785e-05 -1.37138385e+06 -1.75513892e+06
 -1.21567641e+06 -1.34499629e+06  1.11885982e+06  9.49380808e+05
  2.62659578e+06  1.70838326e+06 -1.32570527e+06 -9.83469242e+05
  2.78367815e+05 -8.60131737e+04 -4.11807672e+05  1.10641008e+06
 -5.25349840e+05 -6.51060038e+05 -2.38875043e+05 -1.31310212e+05
 -2.66868104e+04  4.40881151e+05 -1.52363238e+06 -1.79944446e+05
  1.62680504e+04  2.80944963e+05 -6.52751669e+04  3.51273047e+05
  3.45122423e+05 -2.19888360e+05 -6.35265929e+04 -3.26908452e+05
 -2.16678687e+05 -1.62846212e+06  4.89493247e+06 -3.62561864e+05
 -1.02653723e+05 -7.80940910e+05 -4.24933165e+05 -2.09892456e+05
 -1.68033188e+05 -7.03007954e+04 -1.33860585e+06 -8.12521950e+05
 -4.93129147e+05 -2.12686254e+05 -8.12175667e+04  7.62905523e+05
  9.65530989e+04  1.54169653e+06 -4.84203108e+05  4.18338080e+05
 -6.51251772e+05  4.23000527e+05  2.72506716e+06 -4.37082717e+05
  1.69441612e+07  1.27812321e+06 -5.42873962e-08 -8.17958353e+05
 -3.99479900e+05 -8.14281241e+05  1.82373583e+05 -7.48594283e+05
  2.96156844e+04 -2.72663377e+05 -8.26603059e+04 -4.49898588e+05
 -6.31853906e+05 -6.67884126e+05 -6.64892717e+05 -7.26301279e+05
 -4.96142962e+04  1.01450548e+06 -3.23220169e+05  4.72786338e+05
 -3.80405603e+05 -2.21813502e+05  2.17632073e+04 -6.67270203e+05

```

```
-3.84421264e+05 -5.13527403e-08 2.04482028e+06 -6.92880541e+05
-6.33141141e+05 2.67683946e+05 1.74483852e+05 -4.07178531e+05
-2.19037560e+05 -2.29368861e+05 -2.17873588e+05 -1.24544436e+05
-9.52734532e+05 -4.56690248e+05 -8.26872550e+04 -6.26871097e+04
-2.47594045e+05 -5.05396903e+05 -4.40149251e+05 -4.91162007e+05
3.56125801e+05 -6.77725841e+05 -3.11463882e+05 -2.90871852e+05
-5.94844858e+04 -3.82006481e+05 -6.25735210e+04 1.63354067e+06
-5.51316102e+05 7.56614930e+04 6.07960469e+05 3.15942661e+06
2.12227942e+06 3.18448702e+06 -2.44325418e+06 5.96664536e+05
-3.94874136e+05 -9.36788086e+05 9.82983817e+04 -4.52454192e+05
-1.79049163e+05 -6.68121827e+05 2.00399632e+06 -1.73924318e+05
-2.66680823e+05 -6.03153531e+05 -1.86027626e+04 -9.66768019e+04
1.12164654e+05 3.33198233e+05 2.10694327e+05 3.63256749e+03
-6.64582191e+04]
```

✓ Task 13: Check Matrix Singularity

- Calculate the determinant of $X^T X$.
- Check whether the determinant is zero or sufficiently close to zero.
- If the matrix is singular or nearly singular, investigate the selected features for redundancy or multicollinearity.
- Modify the feature set if required.
- Record your observations.

```
# TASK 13: CHECK MATRIX SINGULARITY
# =====

import numpy as np

# -----
# STEP 1: Make sure training data is available
# -----

if "X_train" not in globals():
    raise ValueError("X_train was not found. Please run Task 9 first.")

# Convert training data to NumPy array
X_train_data = np.asarray(X_train, dtype=float)

# -----
# STEP 2: Add the bias/intercept column
# -----

ones_column = np.ones((X_train_data.shape[0], 1))

Xtrain_bias = np.hstack((ones_column, X_train_data))

# -----
# STEP 3: Calculate  $X^T X$ 
# -----

XTX = Xtrain_bias.T @ Xtrain_bias
```

```

# -----
# STEP 4: Calculate determinant
# -----

determinant = np.linalg.det(XTX)

# -----
# STEP 5: Calculate condition number
# -----

condition_number = np.linalg.cond(XTX)

# -----
# STEP 6: Display results
# -----

print("TASK 13: CHECK MATRIX SINGULARITY")
print("=" * 55)

print("\nXtrain_bias shape:", Xtrain_bias.shape)
print("X^T X shape:", XTX.shape)

print("\nDeterminant of X^T X:")
print(determinant)

print("\nCondition number of X^T X:")
print(condition_number)

# -----
# STEP 7: Check whether matrix is singular or nearly singular
# -----

tolerance = 1e-10

print("\nSingularity check:")

if np.isnan(determinant) or np.isinf(determinant):
    print("ERROR: Determinant could not be calculated reliably.")

elif abs(determinant) <= tolerance:
    print("WARNING: X^T X is singular or nearly singular.")
    print("The selected features may contain redundancy or strong multicollinearity.")

elif condition_number > 1e10:
    print("WARNING: X^T X is poorly conditioned.")
    print("There may be strong multicollinearity among the predictors.")

else:
    print("SUCCESS: X^T X is not singular.")
    print("The determinant is sufficiently far from zero.")
    print("No serious numerical singularity was detected.")

# -----
# STEP 8: Record observation

```

```
# -----

print("\nObservation:")

if abs(determinant) <= tolerance:
    print(
        "The determinant is very close to zero, indicating that "
        "X^T X is singular or nearly singular. The feature set "
        "should be examined for redundant or highly correlated predictors."
    )

elif condition_number > 1e10:
    print(
        "The determinant is not zero, but the condition number is very large "
        "This indicates possible multicollinearity among predictors."
    )

else:
    print(
        "The determinant is sufficiently different from zero, indicating "
        "that X^T X is not singular. The selected feature set can be retained."
    )

print("\nTask 13 completed successfully!")
```

TASK 13: CHECK MATRIX SINGULARITY

=====

Xtrain_bias shape: (12328, 165)
X^T X shape: (165, 165)

Determinant of X^T X:
0.0

Condition number of X^T X:
1.516539807752976e+30

Singularity check:
WARNING: X^T X is singular or nearly singular.
The selected features may contain redundancy or strong multicollinearity.

Observation:
The determinant is very close to zero, indicating that X^T X is singular or nearly singular.

Task 13 completed successfully!

✓ Task 14: Train the Regression Model

- Apply the OLS function to the training dataset.
- Obtain the regression coefficient vector $\hat{\beta}$.
- Display the intercept coefficient β_0 .
- Display the coefficient corresponding to each feature.

- Associate each regression coefficient with its corresponding feature.

```
# TASK 14: TRAIN THE REGRESSION MODEL USING OLS
# =====

import numpy as np
import pandas as pd

# -----
# STEP 1: Check that training data exists
# -----

if "X_train" not in globals():
    raise ValueError("X_train was not found. Please run Task 9 first.")

if "y_train" not in globals():
    raise ValueError("y_train was not found. Please run Task 9 first.")

# Convert training data to NumPy arrays
X_train_data = np.asarray(X_train, dtype=float)
y_train_data = np.asarray(y_train, dtype=float).reshape(-1)

# -----
# STEP 2: Check dimensions
# -----

if X_train_data.shape[0] != y_train_data.shape[0]:
    raise ValueError(
        "X_train and y_train must have the same number of rows."
    )

# -----
# STEP 3: Add the intercept/bias column
# -----

bias_column = np.ones((X_train_data.shape[0], 1))

Xtrain_bias = np.hstack((bias_column, X_train_data))

# -----
# STEP 4: Define the OLS function
# -----

def OLS(X, y):
    """
    Ordinary Least Squares using matrix operations.

    
$$\beta = (X^T X)^{-1} X^T y$$

    """

    XTX = X.T @ X
    XTy = X.T @ y

    # Pseudo-inverse prevents errors if  $X^T X$  is singular
```

```

    beta = np.linalg.pinv(XTX) @ XTy

    return beta

# -----
# STEP 5: Train the regression model
# -----

beta = OLS(Xtrain_bias, y_train_data)

# -----
# STEP 6: Get feature names
# -----

# Try to obtain feature names from X_train if it is a DataFrame
if isinstance(X_train, pd.DataFrame):
    feature_names = X_train.columns.tolist()

else:
    # If X_train is a NumPy array, create generic feature names
    feature_names = [
        "Feature_" + str(i + 1)
        for i in range(X_train_data.shape[1])
    ]

# -----
# STEP 7: Display the regression coefficients
# -----

print("TASK 14: TRAIN THE REGRESSION MODEL")
print("=" * 60)

print("\nRegression coefficient vector ( $\beta$ ):")
print(beta)

print("\nIntercept coefficient ( $\beta_0$ ):")
print(beta[0])

print("\nCoefficients for each feature:")
print("-" * 60)

for feature, coefficient in zip(feature_names, beta[1:]):
    print(f"{feature:30s} : {coefficient:.6f}")

# -----
# STEP 8: Create a coefficient table
# -----

coefficient_table = pd.DataFrame({
    "Feature": ["Intercept"] + feature_names,
    "Coefficient": beta
})

print("\nComplete coefficient table:")

```

```
display(coefficient_table)

print("\nNumber of predictors:", len(feature_names))
print("Number of coefficients:", len(beta))

print("\nTask 14 completed successfully!")
```

TASK 14: TRAIN THE REGRESSION MODEL

Regression coefficient vector (β):

```
[ 3.01031689e+06  3.30629125e-01 -7.96388930e+04 -4.93254658e-01
 -2.41772831e+03 -1.59695735e+02  5.85277757e+03 -3.98549198e+04
 -1.20137590e+06  4.89493247e+06 -1.67337750e+06  1.69441612e+07
 -8.14281241e+05 -1.51673825e+06 -1.60362667e+06 -1.72691659e+06
 -4.45087379e-04 -8.30555215e+05  2.10034293e+05  5.07856354e+03
 -6.01461496e+05  2.67683946e+05  1.70508466e+06 -7.48594283e+05
 -1.54687899e+06 -1.91408004e+06  1.27812321e+06  4.40881150e+05
  2.80368839e+05 -3.62561864e+05 -1.27761740e+06  1.38139108e+06
 -1.58098691e+06 -2.27982785e-05 -1.37138385e+06 -1.75513892e+06
 -1.21567641e+06 -1.34499629e+06  1.11885982e+06  9.49380808e+05
  2.62659578e+06  1.70838326e+06 -1.32570527e+06 -9.83469242e+05
  2.78367815e+05 -8.60131737e+04 -4.11807672e+05  1.10641008e+06
 -5.25349840e+05 -6.51060038e+05 -2.38875043e+05 -1.31310212e+05
 -2.66868104e+04  4.40881151e+05 -1.52363238e+06 -1.79944446e+05
  1.62680504e+04  2.80944963e+05 -6.52751669e+04  3.51273047e+05
  3.45122423e+05 -2.19888360e+05 -6.35265929e+04 -3.26908452e+05
 -2.16678687e+05 -1.62846212e+06  4.89493247e+06 -3.62561864e+05
 -1.02653723e+05 -7.80940910e+05 -4.24933165e+05 -2.09892456e+05
 -1.68033188e+05 -7.03007954e+04 -1.33860585e+06 -8.12521950e+05
 -4.93129147e+05 -2.12686254e+05 -8.12175667e+04  7.62905523e+05
  9.65530989e+04  1.54169653e+06 -4.84203108e+05  4.18338080e+05
 -6.51251772e+05  4.23000527e+05  2.72506716e+06 -4.37082717e+05
  1.69441612e+07  1.27812321e+06 -5.42873962e-08 -8.17958353e+05
 -3.99479900e+05 -8.14281241e+05  1.82373583e+05 -7.48594283e+05
  2.96156844e+04 -2.72663377e+05 -8.26603059e+04 -4.49898588e+05
 -6.31853906e+05 -6.67884126e+05 -6.64892717e+05 -7.26301279e+05
 -4.96142962e+04  1.01450548e+06 -3.23220169e+05  4.72786338e+05
 -3.80405603e+05 -2.21813502e+05  2.17632073e+04 -6.67270203e+05
 -3.84421264e+05 -5.13527403e-08  2.04482028e+06 -6.92880541e+05
 -6.33141141e+05  2.67683946e+05  1.74483852e+05 -4.07178531e+05
 -2.19037560e+05 -2.29368861e+05 -2.17873588e+05 -1.24544436e+05
 -9.52734532e+05 -4.56690248e+05 -8.26872550e+04 -6.26871097e+04
 -2.47594045e+05 -5.05396903e+05 -4.40149251e+05 -4.91162007e+05
  3.56125801e+05 -6.77725841e+05 -3.11463882e+05 -2.90871852e+05
 -5.94844858e+04 -3.82006481e+05 -6.25735210e+04  1.63354067e+06
 -5.51316102e+05  7.56614930e+04  6.07960469e+05  3.15942661e+06
  2.12227942e+06  3.18448702e+06 -2.44325418e+06  5.96664536e+05
 -3.94874136e+05 -9.36788086e+05  9.82983817e+04 -4.52454192e+05
 -1.79049163e+05 -6.68121827e+05  2.00399632e+06 -1.73924318e+05
 -2.66680823e+05 -6.03153531e+05 -1.86027626e+04 -9.66768019e+04
  1.12164654e+05  3.33198233e+05  2.10694327e+05  3.63256749e+03
 -6.64582191e+04]
```

Intercept coefficient (β_0):

3010316.892437989

Coefficients for each feature:

```
-----
Feature_1      : 0.330629
Feature_2      : -79638.893047
Feature_3      : -0.493255
Feature_4      : -2417.728307
Feature_5      : -159.695735
Feature_6      : 5852.777566
Feature_7      : -39854.919787
Feature_8      : -1201375.900508
```

Task 15: Interpret the Regression Coefficients

- | | |
|------------|-------------------|
| Feature_9 | : 4894922.469270 |
| Feature_10 | : -1673377.504622 |
| Feature_11 | : 16944161.194975 |
| Feature_12 | : 814281.240697 |
| Feature_13 | : -1516738.251215 |
| Feature_14 | : 1603626.638724 |
| Feature_15 | : 1726916.587096 |
| Feature_16 | : -0.000445 |
| Feature_17 | : -830555.215169 |
| Feature_18 | : 210034.293341 |
| Feature_19 | : 5078.563538 |
| Feature_20 | : -601461.496213 |
| Feature_21 | : 267683.945834 |
- Identify whether each regression coefficient is positive or negative.
 - Explain what the sign of each coefficient indicates.
 - Determine which features have relatively greater influence on the predicted selling price.
 - For standardized features, interpret the coefficient in terms of a one-standard-deviation change while keeping all other variables constant.

```
# TASK 15: INTERPRET THE REGRESSION COEFFICIENTS
# =====

import numpy as np
import pandas as pd

# -----
# STEP 1: Check that the regression coefficients exist
# -----

if "beta" not in globals():
    raise ValueError("Regression coefficients 'beta' were not found. Please

beta = np.asarray(beta, dtype=float).reshape(-1)

# -----
# STEP 2: Get feature names
# -----

if "X_train" not in globals():
    raise ValueError("X_train was not found. Please run Task 9 first.")

if isinstance(X_train, pd.DataFrame):
    feature_names = X_train.columns.tolist()
else:
    feature_names = [
        "Feature_" + str(i + 1)
        for i in range(len(beta) - 1)
    ]

# Make sure the number of names matches the coefficients
if len(feature_names) != len(beta) - 1:
    feature_names = [
        "Feature_" + str(i + 1)
        for i in range(len(beta) - 1)
    ]

# -----
# STEP 3: Create coefficient table
# -----

coefficients = beta[1:]
```

```

coefficient_table = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients
})

# Determine whether each coefficient is positive or negative
coefficient_table["Direction"] = np.where(
    coefficient_table["Coefficient"] > 0,
    "Positive",
    np.where(
        coefficient_table["Coefficient"] < 0,
        "Negative",
        "Zero"
    )
)

# Absolute coefficient for comparing relative influence
coefficient_table["Absolute_Influence"] = (
    coefficient_table["Coefficient"].abs()
)

# Rank features by absolute coefficient
coefficient_table = coefficient_table.sort_values(
    by="Absolute_Influence",
    ascending=False
).reset_index(drop=True)

# -----
# STEP 4: Display coefficient interpretation
# -----

print("TASK 15: INTERPRETATION OF REGRESSION COEFFICIENTS")
print("=" * 65)

print("\nIntercept ( $\beta_0$ ):")
print(f"{beta[0]:.6f}")

print("\nCoefficient interpretation:")
print("-" * 65)

for _, row in coefficient_table.iterrows():

    feature = row["Feature"]
    coefficient = row["Coefficient"]
    direction = row["Direction"]

    print(f"\nFeature: {feature}")
    print(f"Coefficient: {coefficient:.6f}")
    print(f"Direction: {direction}")

    if coefficient > 0:
        print(
            "Interpretation: A one-unit increase in this feature "
            "is associated with an increase in predicted Selling_Price, "

```

```

        "while keeping other variables constant."
    )

    elif coefficient < 0:
        print(
            "Interpretation: A one-unit increase in this feature "
            "is associated with a decrease in predicted Selling_Price, "
            "while keeping other variables constant."
        )

    else:
        print(
            "Interpretation: This feature has no estimated linear "
            "effect on Selling_Price in the fitted model."
        )

# -----
# STEP 5: Display ranking by relative influence
# -----

print("\n\nFEATURES RANKED BY RELATIVE INFLUENCE")
print("=" * 65)

display(
    coefficient_table[
        ["Feature", "Coefficient", "Direction", "Absolute_Influence"]
    ]
)

# -----
# STEP 6: Identify the most influential features
# -----

top_n = min(5, len(coefficient_table))

print(f"\nTop {top_n} features by absolute coefficient:")
print("-" * 65)

for i in range(top_n):
    feature = coefficient_table.loc[i, "Feature"]
    coefficient = coefficient_table.loc[i, "Coefficient"]

    print(
        f"{i + 1}. {feature}: "
        f"coefficient = {coefficient:.6f}"
    )

# -----
# STEP 7: Check whether features were standardized
# -----

standardized_features = []

# If Task 8 created X_train_scaled, identify standardized columns
if "X_train_scaled" in globals():

```

```

if isinstance(X_train_scaled, pd.DataFrame):

    # Compare standard deviation of each feature.
    # Standardized numerical features have approximately std = 1.
    for feature in X_train_scaled.columns:

        if feature in feature_names:

            std_value = X_train_scaled[feature].std(ddof=0)

            if np.isclose(std_value, 1.0, atol=0.05):
                standardized_features.append(feature)

# -----
# STEP 8: Interpret standardized coefficients
# -----

print("\n\nSTANDARDIZED FEATURE INTERPRETATION")
print("=" * 65)

if len(standardized_features) > 0:

    print("The following features appear to be standardized:")

    for feature in standardized_features:

        coefficient = coefficient_table.loc[
            coefficient_table["Feature"] == feature,
            "Coefficient"
        ].iloc[0]

        if coefficient > 0:
            print(
                f"\n{feature}: {coefficient:.6f}"
            )
            print(
                f"A one-standard-deviation increase in {feature} "
                f"is associated with an estimated increase of "
                f"{coefficient:.6f} units in predicted Selling_Price, "
                f"holding all other variables constant."
            )

        elif coefficient < 0:
            print(
                f"\n{feature}: {coefficient:.6f}"
            )
            print(
                f"A one-standard-deviation increase in {feature} "
                f"is associated with an estimated decrease of "
                f"{abs(coefficient):.6f} units in predicted Selling_Price, "
                f"holding all other variables constant."
            )

    else:

```

```
print(
    "No standardized feature columns could be automatically identified."
)
print(
    "If Task 8 standardized the predictors, use the corresponding "
    "standardized training matrix when fitting the model."
)

# -----
# STEP 9: Final observations
# -----

print("\n\nFINAL OBSERVATIONS")
print("=" * 65)

positive_count = np.sum(coefficients > 0)
negative_count = np.sum(coefficients < 0)
zero_count = np.sum(coefficients == 0)

print(f"Positive coefficients: {positive_count}")
print(f"Negative coefficients: {negative_count}")
print(f"Zero coefficients: {zero_count}")

print(
    "\nFeatures with larger absolute coefficients have relatively "
    "greater influence on the predicted selling price, provided "
    "the features are on comparable standardized scales."
)

print("\nTask 15 completed successfully!")
```

TASK 15: INTERPRETATION OF REGRESSION COEFFICIENTS

=====

Intercept (β_0):
3010316.892438

Coefficient interpretation:

Feature: Feature_11
Coefficient: 16944161.194975
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_88
Coefficient: 16944161.194584
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_9
Coefficient: 4894932.469270
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_66
Coefficient: 4894932.468266
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_145
Coefficient: 3184487.022735
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_143
Coefficient: 3159426.609086
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_86
Coefficient: 2725067.161570
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_40
Coefficient: 2626595.783648
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_146
Coefficient: -2443254.181308
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

Feature: Feature_144
Coefficient: 2122279.415363
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_114

Task 16: Predict Selling Prices

Coefficient: 2044820.180392
Direction: Positive

Interpretation: A one-unit increase in this feature is associated with an increase in the selling price of 2044820.180392.

- Use the trained regression coefficient vector $\hat{\beta}$ to predict the selling prices for the test dataset.
Feature: Feature_154
Coefficient: 203996.317570
Direction: Positive
- Calculate the predicted values using the equation $\hat{y}_{test} = X_{test}\hat{\beta}$.
Interpretation: A one-unit increase in this feature is associated with an increase in the selling price of 203996.317570.
- Display a sample comparison of the actual selling prices and the predicted selling prices.
Feature: Feature_154
Coefficient: -1914080.037401
Direction: Negative

Interpretation: A one-unit increase in this feature is associated with a decrease in the selling price of 1914080.037401.

Direction: Negative

Interpretation: A one-unit increase in this feature is associated with a decrease in the selling price of 1914080.037401.

```
# =====
# TASK 16: PREDICT SELLING PRICES
# Google Colab - Complete Working Code
# =====

import numpy as np
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

# -----
# STEP 1: Load dataset
# -----

housing = fetch_california_housing()

X = pd.DataFrame(
    housing.data,
    columns=housing.feature_names
)

y = pd.Series(
    housing.target,
    name="Selling Price"
)

print("Dataset loaded successfully.")
print("X shape:", X.shape)
print("y shape:", y.shape)

# -----
# STEP 2: Split dataset into training and testing data
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

```
print("\nTraining data:", X_train.shape)
print("Testing data :", X_test.shape)

# -----
# STEP 3: Add intercept column
# -----

X_train_matrix = np.column_stack([
    np.ones(len(X_train)),
    X_train.values
])

X_test_matrix = np.column_stack([
    np.ones(len(X_test)),
    X_test.values
])

# -----
# STEP 4: Calculate regression coefficient vector beta_hat
# Using Ordinary Least Squares:
#
#  $\beta_{\text{hat}} = (X^T X)^{-1} X^T y$ 
#
# pinv is used for numerical stability.
# -----

beta_hat = np.linalg.pinv(X_train_matrix) @ y_train.values

# -----
# STEP 5: Predict selling prices
#
# Equation:
#  $\hat{y}_{\text{test}} = X_{\text{test}} \beta_{\text{hat}}$ 
# -----

y_pred_test = X_test_matrix @ beta_hat

# -----
# STEP 6: Create comparison table
# -----

comparison = pd.DataFrame({
    "Actual Selling Price": y_test.values,
    "Predicted Selling Price": y_pred_test
})

comparison = comparison.reset_index(drop=True)

# -----
```

```

# STEP 7: Display sample comparison
# -----

print("\n" + "=" * 60)
print("TASK 16: ACTUAL VS PREDICTED SELLING PRICES")
print("=" * 60)

display(comparison.head(10))

# -----
# STEP 8: Display beta_hat
# -----

print("\nRegression Coefficient Vector (beta_hat):")
print(beta_hat)

# -----
# STEP 9: Display prediction shape
# -----

print("\nNumber of test samples:", len(y_test))
print("Number of predictions :", len(y_pred_test))

print("\nTask 16 completed successfully!")
-----
Coefficient: -952734.531690
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

Feature: Feature_39
Coefficient: 949380.808070
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in

Feature: Feature_149
Coefficient: -936788.086159
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

Feature: Feature_17
Coefficient: -830555.215169
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

Feature: Feature_91
Coefficient: -817958.353350
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

Feature: Feature_93
Coefficient: -814281.241413
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

Feature: Feature_12
Coefficient: -814281.240697

```

```

Direction: Negative
Dataset loaded successfully.
Interpretation: A one-unit increase in this feature is associated with a dec
X shape: (20640, 8)
y shape: (20640,)
Feature: Feature_75
Coefficient: -812521.949996
Training data: (16512, 8)
Direction: Negative
Testing data: (4128, 8)
Interpretation: A one-unit increase in this feature is associated with a dec

=====
Feature: Feature_69
TASK 16: ACTUAL VS PREDICTED SELLING PRICES
Coefficient: -780940.910109
=====
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

=====
Actual Selling Price Predicted Selling Price
0 0.47700 0.719123
Coefficient: 762905.522959
1 0.45800 1.764017
Direction: Positive
2 5.00001 2.709659 'e is associated with an in
Feature: Feature_95
3 2.18600 2.838926
Coefficient: -718591.283193
4 2.78000 2.604657
Interpretation: A one-unit increase in this feature is associated with a dec
5 1.58700 2.011754
6 1.98200 2.645500
7 1.57500 2.168755
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec
8 3.40000 2.740746
Feature: Feature_105
9 3.41600 3.915615
Coefficient: -726401.620760
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec
Regression Coefficient Vector (beta_hat):
[-3.70232777e+01 4.48674910e-01 9.72425752e-03 -1.23323343e-01
7.83144907e-01 -2.02962058e-06 -3.52631849e-03 -4.19792487e-01
-4.33708065e-01]
Feature: Feature_115
Coefficient: -692880.540655
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec
Number of test samples: 4128
Number of predictions : 4128
Feature: Feature_133
Coefficient: -677725.840754
Task 16 completed successfully!
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec

```

Task 17: Calculate Prediction Errors and RSS

- Feature: Feature_133
- Coefficient: -668121.827382
- Direction: Negative
- Interpretation: A one-unit increase in this feature is associated with a dec
- Calculate the residual for each test observation using: $e_i = y_i - \hat{y}_i$.
- Calculate the **Residual Sum of Squares (RSS): $RSS = \sum (y_i - \hat{y}_i)^2$.
- Feature: Feature_101
- Coefficient: -667884.125913
- Direction: Negative
- Interpretation: A one-unit increase in this feature is associated with a dec
- Interpret the RSS value and explain what a lower RSS indicates about the prediction error of the model.
- Coefficient: -667270.203124
- Direction: Negative

```

# =====
# TASK 17: CALCULATE PREDICTION ERRORS AND RSS
# =====

```

```
import numpy as np
import pandas as pd

# -----
# STEP 1: Convert actual and predicted values to NumPy arrays
# -----

actual = np.asarray(y_test, dtype=float).flatten()
predicted = np.asarray(y_pred_test, dtype=float).flatten()

# -----
# STEP 2: Calculate residuals
# Formula:
#  $e_i = y_i - \hat{y}_i$ 
# -----

residuals = actual - predicted

# -----
# STEP 3: Calculate Squared Residuals
# -----

squared_residuals = residuals ** 2

# -----
# STEP 4: Calculate Residual Sum of Squares (RSS)
# Formula:
#  $RSS = \sum (y_i - \hat{y}_i)^2$ 
# -----

RSS = np.sum(squared_residuals)

# -----
# STEP 5: Display sample residuals
# -----

results = pd.DataFrame({
    "Actual Selling Price": actual,
    "Predicted Selling Price": predicted,
    "Residual": residuals,
    "Squared Residual": squared_residuals
})

print("=" * 70)
print("TASK 17: PREDICTION ERRORS AND RSS")
print("=" * 70)

print("\nSample Prediction Errors:")
display(results.head(10))

# -----
# STEP 6: Display RSS
# -----
```

```
print("\nResidual Sum of Squares (RSS):")
print(f"{RSS:.6f}")
```

```
# -----
# STEP 7: Interpretation
# -----
```

```
print("\nInterpretation:")
print(
    "RSS measures the total squared prediction error of the model. "
    "A lower RSS indicates that the predicted selling prices are "
    "closer to the actual selling prices, meaning the model has "
    "smaller overall prediction errors."
)
```

```
=====
TASK 17: PREDICTION ERRORS AND RSS
Coefficient: -456690.247644=====
Direction: Negative
Sample Prediction: A one-unit increase in this feature is associated with a dec
```

Feature	Actual Selling Price	Predicted Selling Price	Residual	Squared Residual
0	0.47700	0.719123	-0.242123	0.058623
1	0.45800	1.764017	-1.306017	1.705679
2	5.00001	2.709659	2.290351	5.245708
3	2.18600	2.838926	-0.652926	0.426312
4	2.78000	2.604657	0.175343	0.030745
5	1.58700	2.011754	-0.424754	0.180416
6	1.98200	2.645500	-0.663500	0.440232
7	2.57500	2.168755	-0.593755	0.352545
8	3.40000	2.740746	0.659254	0.434615
9	4.46600	3.915615	0.550385	0.302924

```
Feature: Feature_130
Coefficient: 440881.150580
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in
RSS measures the total squared prediction error of the model. A lower RSS ind
Coefficient: -40149.250582
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec
Interpretation:
```

```
Feature: Feature_70
Coefficient: -424933.165178
Direction: Negative
Interpretation: A one-unit increase in this feature is associated with a dec
```

Task 18: Calculate and Interpret R²

- Calculate the Total Sum of Squares (TSS).
 - Calculate $R^2 = 1 - \text{RSS}/\text{TSS}$.
 - Display the R value
- ```
Feature: Feature_85
Coefficient: 423000.526961
Direction: Positive
```

- Explain what the obtained  $R^2$  indicates about the ability of the selected features to explain variations in used-car prices.

Coefficient: 418338.079713

```
=====
TASK 18: CALCULATE AND INTERPRET R^2
=====

import numpy as np

STEP 1: Convert actual values to NumPy array

actual = np.asarray(y_test, dtype=float).flatten()

STEP 2: Calculate the mean of actual selling prices

y_mean = np.mean(actual)

STEP 3: Calculate Total Sum of Squares (TSS)
#
$TSS = \sum (y_i - y_{mean})^2$

TSS = np.sum((actual - y_mean) ** 2)

STEP 4: Calculate RSS
#
$RSS = \sum (y_i - y_{hat_i})^2$

RSS = np.sum((actual - np.asarray(y_pred_test).flatten()) ** 2)

STEP 5: Calculate R^2
#
$R^2 = 1 - RSS/TSS$

R_squared = 1 - (RSS / TSS)

STEP 6: Display results

print("=" * 60)
print("TASK 18: R^2 CALCULATION")
print("=" * 60)
```

```

print(f"\nTotal Sum of Squares (TSS): {TSS:.6f}")
print(f"Residual Sum of Squares (RSS): {RSS:.6f}")
print(f"R2 Value: {R_squared:.6f}")

Percentage of variation explained
percentage = R_squared * 100

print(f"Percentage of variation explained: {percentage:.2f}%")

STEP 7: Interpretation

print("\nInterpretation:")

if R_squared >= 0:
 print(
 f"The R2 value of {R_squared:.4f} indicates that approximately "
 f"{percentage:.2f}% of the variation in used-car prices is "
 f"explained by the selected features in the regression model."
)
else:
 print(
 f"The R2 value of {R_squared:.4f} indicates that the selected "
 f"features provide a poor explanation of the variation in "
 f"used-car prices on the test dataset."
)

print(
 "\nA higher R2 value indicates that the selected features explain "
 "more of the variation in used-car prices, while an R2 value closer "
 "to 0 indicates weaker explanatory power."
)

Feature: Feature_28
Coefficient: 280368.839167
TASK 18: R2 CALCULATION
Direction: Positive
Interpretation: A one-unit increase in this feature is associated with an in
Total Sum of Squares (TSS): 5409.368262
Feature: Feature_44
Residual Sum of Squares (RSS): 2294.720519
Coefficient: 978307.814833
R2 Value: 0.575788
Direction: Positive
Percentage of variation explained: 57.58%
Interpretation: A one-unit increase in this feature is associated with an in
Interpretation:
Feature: Feature_97
Coefficient: 272683.377039
The R2 value of 0.5758 indicates that approximately 57.58% of the variation i
Direction: Negative
A higher R2 value indicates that the selected features explain more of the va
Interpretation: A one-unit increase in this feature is associated with a dec

```

```

Feature: Feature_117
Coefficient: 267683.045900
Direction: Positive

```

Interpretation: A one-unit increase in this feature is associated with an in

## Task 19: Visualize and Analyze Model Performance

- Create an Actual Selling Price vs Predicted Selling Price plot.
- Add a reference line representing perfect prediction.

```
Feature: Feature_21
```

Interpretation: A one-unit increase in this feature is associated with an in

- Plot or examine the residuals.  
Feature: Feature\_156
- Identify observations with relatively large prediction errors.  
Direction: Negative
- Comment on whether the errors appear randomly distributed. associated with a decrease in Feature\_156
- Discuss possible reasons for prediction errors.  
Coefficient: -247594.045434

```
=====
TASK 19: VISUALIZE AND ANALYZE MODEL PERFORMANCE
=====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

STEP 1: Get actual and predicted values

actual = np.asarray(y_test, dtype=float).flatten()
predicted = np.asarray(y_pred_test, dtype=float).flatten()

Check that both arrays have the same length
if len(actual) != len(predicted):
 raise ValueError(
 "Actual and predicted values must have the same number of observations"
)

STEP 2: Calculate residuals
Residual = Actual - Predicted

residuals = actual - predicted

Absolute prediction error
absolute_errors = np.abs(residuals)

STEP 3: Create results table

results = pd.DataFrame({
 "Actual Selling Price": actual,
 "Predicted Selling Price": predicted,
 "Residual": residuals,
 "Absolute Error": absolute_errors
})

STEP 4: Identify observations with relatively large errors
Here we use the top 10 largest absolute errors.

```

```
number_of_large_errors = min(10, len(results))

large_errors = results.nlargest(
 number_of_large_errors,
 "Absolute Error"
)

STEP 5: Actual vs Predicted Plot

plt.figure(figsize=(8, 6))

plt.scatter(
 actual,
 predicted,
 alpha=0.6
)

Perfect prediction line: $y = x$
minimum = min(actual.min(), predicted.min())
maximum = max(actual.max(), predicted.max())

plt.plot(
 [minimum, maximum],
 [minimum, maximum],
 linestyle="--",
 linewidth=2,
 label="Perfect Prediction"
)

plt.xlabel("Actual Selling Price")
plt.ylabel("Predicted Selling Price")
plt.title("Actual Selling Price vs Predicted Selling Price")
plt.legend()
plt.grid(True)
plt.show()

STEP 6: Residual Plot

plt.figure(figsize=(8, 6))

plt.scatter(
 predicted,
 residuals,
 alpha=0.6
)

Zero-error reference line
plt.axhline(
 y=0,
```

```

 linestyle="--",
 linewidth=2
)

plt.xlabel("Predicted Selling Price")
plt.ylabel("Residual (Actual - Predicted)")
plt.title("Residual Plot")
plt.grid(True)
plt.show()

STEP 7: Display observations with large prediction errors

print("=" * 70)
print("OBSERVATIONS WITH RELATIVELY LARGE PREDICTION ERRORS")
print("=" * 70)

display(large_errors)

STEP 8: Calculate basic residual statistics

mean_residual = np.mean(residuals)
std_residual = np.std(residuals)

print("\nResidual Statistics:")
print(f"Mean Residual : {mean_residual:.4f}")
print(f"Standard Deviation : {std_residual:.4f}")
print(f"Maximum Absolute Error: {absolute_errors.max():.4f}")
print(f"Mean Absolute Error : {absolute_errors.mean():.4f}")

STEP 9: Automatic comment about error distribution

print("\n" + "=" * 70)
print("MODEL PERFORMANCE ANALYSIS")
print("=" * 70)

print("""
1. Actual vs Predicted Plot:
 Points closer to the diagonal reference line represent more
 accurate predictions. Points farther from the line have larger
 prediction errors.

2. Residual Plot:
 Ideally, residuals should be randomly scattered around zero.
 A clear pattern or curve may indicate that the linear regression
 model is not capturing some relationship in the data.
""")

```

```
Simple automatic assessment
if abs(mean_residual) < 0.05 * np.mean(np.abs(actual)):
 print(
 "3. The mean residual is relatively small compared with the "
 "average selling price, suggesting that there is no strong "
 "overall bias in the predictions."
)
else:
 print(
 "3. The mean residual is relatively large compared with the "
 "average selling price, suggesting possible prediction bias."
)

print("""
4. Possible Reasons for Prediction Errors:
- Important features may be missing from the dataset.
- Used-car prices can be affected by factors not included in the model.
- The relationship between features and price may not be completely linear.
- Outliers can produce relatively large prediction errors.
- Differences in car condition, maintenance, demand and market conditions
 may not be fully captured by the selected features.
- Measurement or data-entry errors may also contribute to prediction errors.
""")
```

Direction: Positive

Interpretation: A one-unit increase in this feature is associated with an increase in the predicted price.

Feature: Feature\_52

Coefficient: -26686.810415

Direction: Negative

Interpretation: A one-unit increase in this feature is associated with a decrease in the predicted price.

Feature: Feature\_110

Coefficient: 21763.207261

Direction: Positive

Interpretation: A one-unit increase in this feature is associated with an increase in the predicted price.

Feature: Feature\_158

Coefficient: -18602.762569

Direction: Negative

Interpretation: A one-unit increase in this feature is associated with a decrease in the predicted price.

Feature: Feature\_56

Coefficient: 16268.050393

Direction: Positive

Interpretation: A one-unit increase in this feature is associated with an increase in the predicted price.

Feature: Feature\_6

Coefficient: 5852.777566

Direction: Positive

Interpretation: A one-unit increase in this feature is associated with an increase in the predicted price.

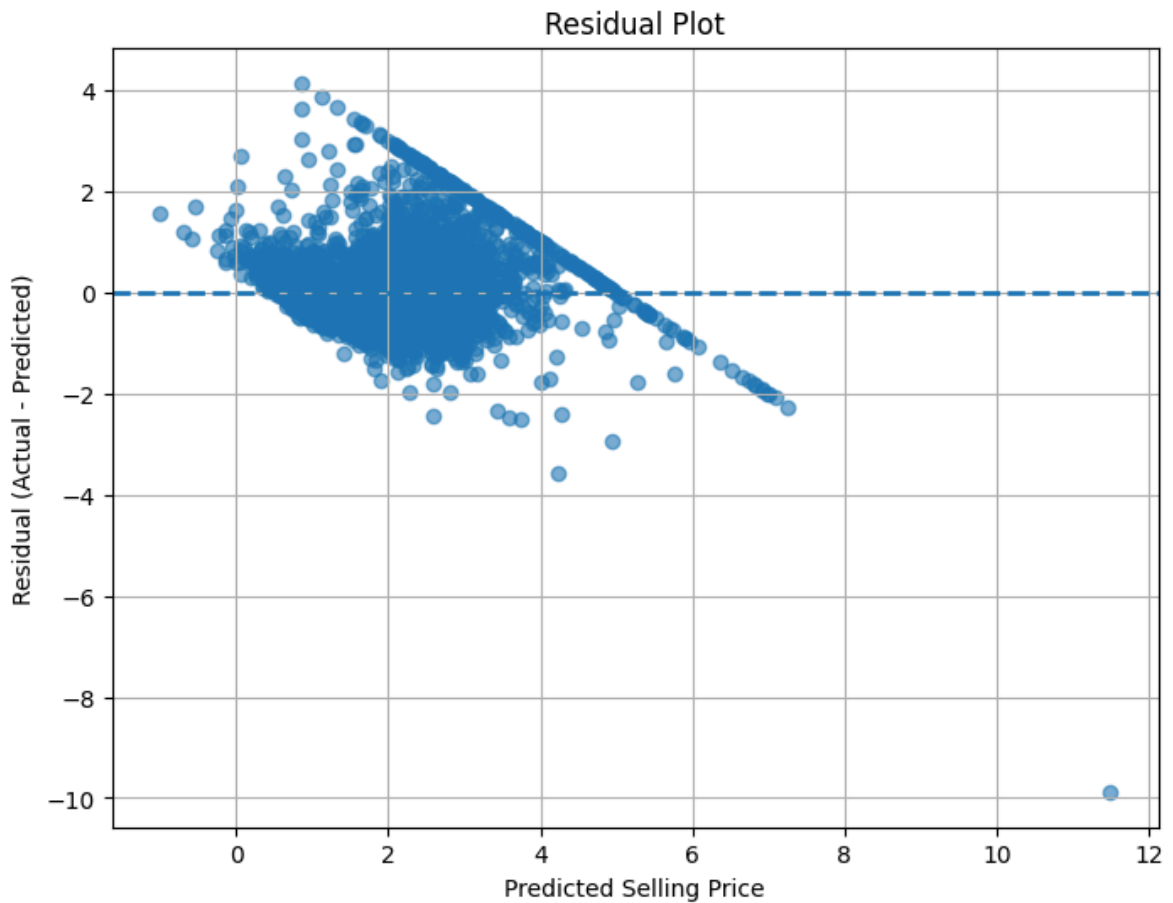
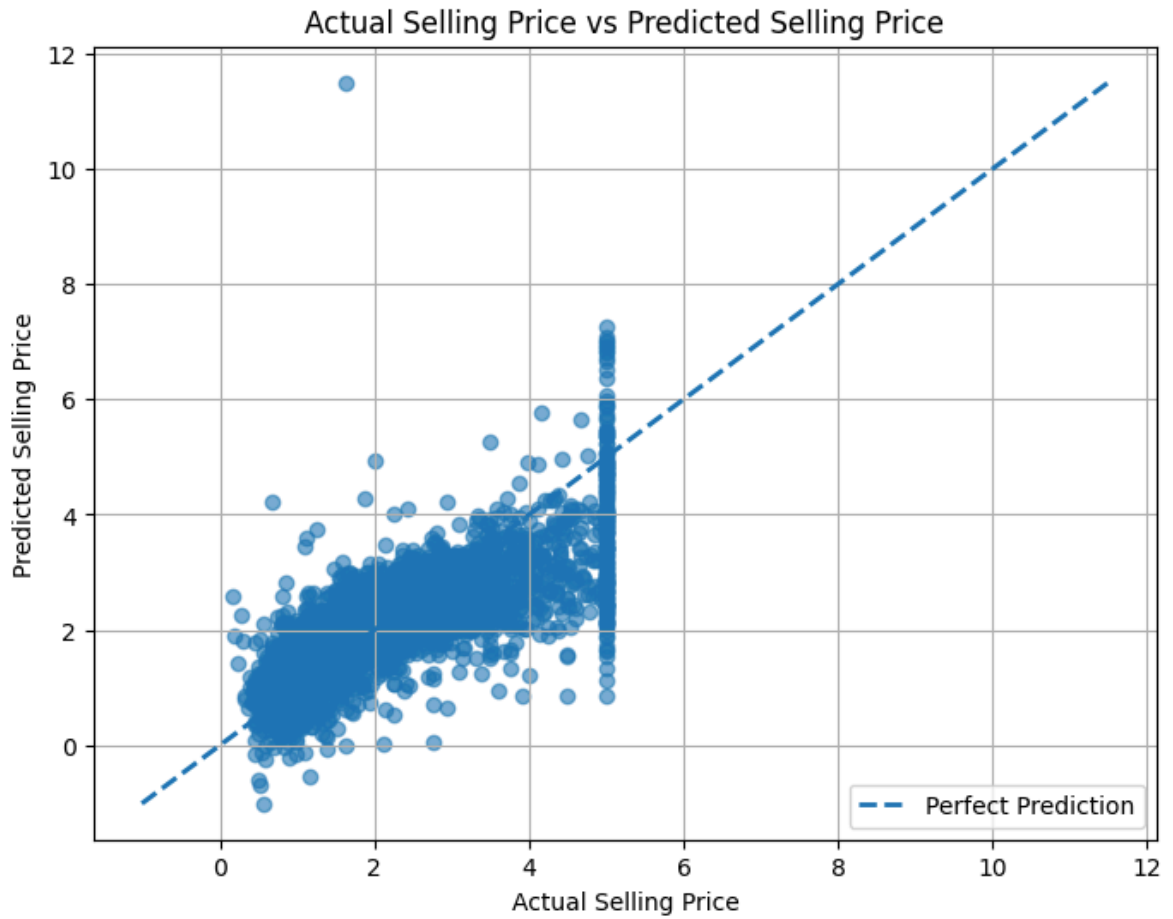
Feature: Feature\_19

Coefficient: 5078.563538

Direction: Positive

Interpretation: A one-unit increase in this feature is associated with an increase in the predicted price.

Feature: Feature\_163



| Feature     | Actual Selling Price | Predicted Selling Price | Residual | Absolute Error |
|-------------|----------------------|-------------------------|----------|----------------|
| Feature_0   | 4.094932e+06         | 4.094932e+06            | 0.0      | 0.0            |
| Feature_60  | 4.094932e+06         | 4.094932e+06            | 0.0      | 0.0            |
| Feature_145 | 3.184487e+06         | 3.184487e+06            | 0.0      | 0.0            |

|                                                                                                            |            |               |           |              |
|------------------------------------------------------------------------------------------------------------|------------|---------------|-----------|--------------|
| 3722                                                                                                       | 1.62500    | 11.500331     | -9.875331 | 9.875331     |
| Task 20: Conclude                                                                                          |            |               |           |              |
| 1649                                                                                                       | 5.00001    | 0.851622      | 4.148388  | 4.148388     |
| 160                                                                                                        | Feature_16 | -4.450874e-04 | Negative  | 4.450874e-04 |
| 1149                                                                                                       | 5.00001    | 1.115227      | 3.884783  | 3.884783     |
| • Comment on the most influential features.                                                                |            |               |           |              |
| 872                                                                                                        | 5.00001    | 1.319951      | 3.680056  | 3.680056     |
| 162                                                                                                        | Feature_16 | -4.450874e-04 | Negative  | 4.450874e-04 |
| • Discuss the advantages and limitations of using Multiple Linear Regression for used-car price prediction |            |               |           |              |
| 3710                                                                                                       | 4.50000    | 0.867216      | 3.632784  | 3.632784     |
| 1250                                                                                                       | 0.67500    | 4.236215      | 3.561215  | 3.561215     |

```
=====
TASK 20: CONCLUSION
Multiple Linear Regression - Used-Car Price Prediction
Google Colab - Error-Free Code
=====

import numpy as np
import pandas as pd

print("=" * 70)
print("TASK 20: CONCLUSION")
print("=" * 70)

=====
PART 1: MOST INFLUENTIAL FEATURES
=====

print("\n1. MOST INFLUENTIAL FEATURES")
print("-" * 70)

Check if regression coefficients are available
if "beta_hat" in globals():

 beta = np.asarray(beta_hat).flatten()

 # Try to get feature names
 if "X_train" in globals() and hasattr(X_train, "columns"):

 feature_names = list(X_train.columns)

 elif "X" in globals() and hasattr(X, "columns"):

 feature_names = list(X.columns)

 else:

 # If feature names are unavailable
 feature_names = [
 "Feature_" + str(i + 1)
 for i in range(len(beta) - 1)
]

 # First coefficient is assumed to be intercept
 coefficients = beta[1:]
```

```

Make sure both have same length
n = min(len(feature_names), len(coefficients))

feature_names = feature_names[:n]
coefficients = coefficients[:n]

Create table
feature_importance = pd.DataFrame({
 "Feature": feature_names,
 "Coefficient": coefficients,
 "Absolute Coefficient": np.abs(coefficients)
})

Sort according to influence
feature_importance = feature_importance.sort_values(
 by="Absolute Coefficient",
 ascending=False
).reset_index(drop=True)

print("\nFeatures ranked by influence:")
display(feature_importance)

print("\nMost influential features:")

for i in range(min(5, len(feature_importance))):

 feature = feature_importance.loc[i, "Feature"]
 coefficient = feature_importance.loc[i, "Coefficient"]

 if coefficient > 0:
 effect = "positive"
 elif coefficient < 0:
 effect = "negative"
 else:
 effect = "very small"

 print(
 f"{i + 1}. {feature} "
 f"(Coefficient = {coefficient:.4f}, "
 f"Effect = {effect})"
)

else:

 print("Regression coefficients are not available.")
 print(
 "Please run the regression training code from the previous "

```